An RM-ODP Based Ontology and a CAD Tool for Modeling Hierarchical Systems in Enterprise Architecture

Lam-Son Lê, Alain Wegmann
School of Computer and Communication Sciences
EPFL
CH-1015 Lausanne, Switzerland
{LamSon.Le, Alain.Wegmann}@epfl.ch

Abstract

Enterprise Architecture (EA) requires modeling enterprise across multiple levels (from markets down to IT systems). Reference Model of Open Distributed System (RM-ODP) was originally defined for describing object-oriented distributed systems but can also be suited to general modeling. To be able to build a Computer Aided Design (CAD) tool based on RM-ODP particularly for modeling hierarchical systems in EA, we need to have a formal description of the terms defined in RM-ODP. In this paper, we present how Part 2 and Part3 of RM-ODP can be integrated to define a computer-interpretable ontology that can be formalized using a declarative language such as Alloy. Thanks to this, our CAD tool can be implemented to allow the modeler to visually build enterprise models of hierarchical systems. The tool can also render Unified Modeling Language (UML) diagrams to show particular aspects of the hierarchical systems.

Keywords: RM-ODP, Enterprise Architecture, Ontology, Formalization, CAD tool.

1. Introduction

The goal of enterprise architecture [1] is to align the business systems and the IT systems in order to improve an enterprise's competitiveness. Enterprise architecture (EA) deals with hierarchical systems that typically span from business entities (market, company, department…) down to IT entities (e.g. applications, applets, servlets, J2EE beans, COM…). Our goal is the development of an EA design method called SEAM (Systemic Enterprise Architecture Methodology) [1] and of the corresponding tools. SEAM is based on Reference Model of Open Distributed Processing (RM-ODP) [2] and is using a UML-like notation. In this paper, we present how the SEAM modeling ontology was developed by combining RM-ODP Part 2 (i.e. Foundations) and Part 3 (i.e. Architecture) and by formalizing the result of this combination. The originality of this paper lies in the application of RM-ODP in the context of EA. This leads to a combination of Part 2 and Part 3 that might be considered as original. We also present how UML-like diagrams can be generated from models developed with this ontology. The snapshots used in the paper are made with our tool that implements this ontology.

The "Reference Model - Open Distributed Processing" (RM-ODP) [2] is an ISO/ITU standard approved in 1996. It provides the definitions and relations between concepts useful to describe object-oriented distributed systems. It positions itself as a "meta-standard" for object-oriented modeling standards. The Object Management Group community adopted in 1998 this standard as a base for describing CORBA systems. RM-ODP has four parts: Part 1 is non-normative and introduces the standard. Part 2 defines the foundations. Part 3 describes the viewpoints necessary to design an IT system. Part 4 is a formalization of the RM-ODP concepts. RM-ODP defines the concepts in a narrative way.

To be able to build a Computer Aided Design (CAD)[1] tool based on RM-ODP particularly for modeling hierarchical systems in EA, we need to have a formal description of the terms defined in RM-ODP. This work was done for RM-ODP Part 2, using the specification language Alloy [3] and the results were published in [4] and [5]. However, even if the terms in Part 2 are formalized in Alloy, this is not enough to build a CAD tool. First, in EA, the concept of hierarchical system modeling is crucial. To be able to build an EA CAD tool, it is important to define how the terms defined in Part 2 can be used to represent hierarchical systems. This is why we need to combine them with Part 3 that defines the viewpoints. Once the terms in Part 2 and Part 3 have been combined and formalized, it is possible to build a CAD tool based on this formalization. Then, the CAD tool can

[1] We call a visual modeling tool for hierarchical systems in EA a CAD tool because its scope is mainly towards marketing and business process modeling rather than software modeling (even if software modeling is possible).

generate UML-like diagrams by filtering the elements to be shown.

Section 2 discusses the integration of RM-ODP Part 2 and Part 3. Section 3 gives the formalization of our ontology in Alloy. Section 4 presents how UML-like diagrams can be generated from the model defined using our ontology. Section 5 outlines the related works.

2. Model Elements Defined by Integrating Viewpoints and Modeling Concepts of RM-ODP

In this section, we explain which viewpoints are necessary to make EA models (Section 2.1) and which model elements are defined in the different viewpoints (Section 2.2). Section 2.1 is based on RM-ODP Part 3 (i.e. Architecture) and Section 2.2 is based on RM-ODP Part 2.

2.1. Viewpoints and Model Elements in EA

EA has requirements that are different from regular IT system analysis and design. In EA, the development teams deal with hierarchical systems that spans from business down to IT. For example, in an EA project implementing a new way to manage price updates in a nation-wide department store required the development of an enterprise with ten hierarchical levels (from market down to Java programming classes). In a regular IT system specification, fewer levels are considered and are dealt with in a less systematic way. In EA, the teams work with an enterprise models that represent all relevant aspects of the company. Our goal is to define a way to build such hierarchical enterprise models that represent all levels in a systematic way. Thanks to this, the different specialists, who are in charge of the different levels, can share a common way to reason about the enterprise model (because all levels represent systems) while having level-specific heuristics to guide their design decision (an organization with people is essentially not designed in a same way as a software is).

Our goal is model "general systems" (regardless of the fact that they are IT related or business related). The main characteristic of the systems is that they can be considered as wholes or as composite. Systems represented as whole exhibit emergent properties. Systems represented as composite expose their construction. In RM-ODP Part 3, the information viewpoints are defined to describe systems considered as atomic (or as wholes); computational viewpoints are defined to describe non-atomic systems (or composite systems). In an information viewpoint, there are information objects. These information objects are useful to describe the system properties. In a computational viewpoint, there are component computational objects that describe the sub-systems. So, in SEAM, we consider the computational viewpoint and the information viewpoint are the main viewpoints.

As we model hierarchical system, we believe that it is important to make explicit which system is described by these viewpoints, so we always consider that we have information viewpoints of computational objects and computational viewpoints of computational objects. Let's illustrate this by an example. We model a group of companies collaborating to serve a customer. We call this group of companies a supplier value network. The supplier value network can be considered as a computational object. The information viewpoint of this computational object describes the customer perception of this supplier value network. Figure 1 is a snapshot of our CAD tool [6] that shows the supplier value network as a computational object. "SellTxn" and "Sell" are the main information object and the main localized action of the supplier, respectively.

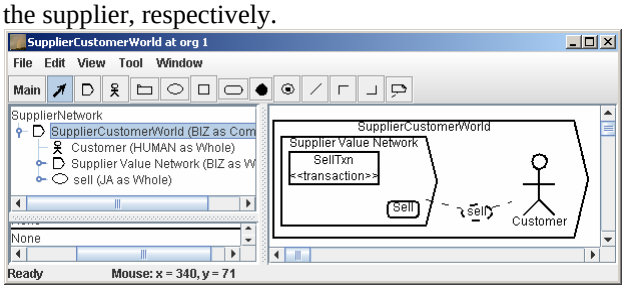

Figure 1. Information viewpoint of the computational object "supplier value network"

The computational viewpoint of this computational object describes which company collaborates to serve the customer. Figure 2 is another snapshot showing 3 companies collaborating within the supplier. Note that, in the computational viewpoint of the supplier, each company is described in terms of its information viewpoint.

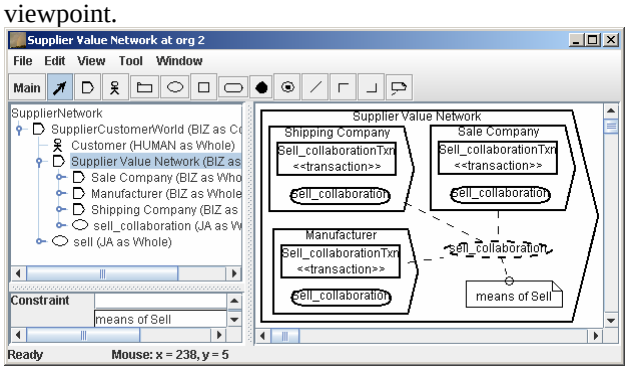

Figure 2. Computational viewpoint of the computational object "supplier value network"

In summary our EA models are composed of a series of levels, called organizational levels, in which computational objects are represented. These

computational objects can be either specified by their information viewpoints or by their computational viewpoints.

RM-ODP defines five viewpoints to represent IT systems; enterprise, information, computational, engineering and technology viewpoints. We have discussed our use of the information and computational viewpoints which are central in SEAM. We analyze now the other viewpoints.

In SEAM we do not use the enterprise viewpoints. When designing an IT system, it is important to represent the environment in which the IT system exists. In SEAM, this is done through modeling of the organizational levels which are above the IT system: e.g. department level, company level or market level.

The engineering viewpoints and the technology viewpoints can be found in each organizational level. They are used to define the behavior templates that will have to be implemented in each level. For example, in an EA model, in the organizational level that represent people, the engineering viewpoint and the technology viewpoint are used to generate job descriptions. In the organizational level that describes Java programs, the engineering and technology viewpoints are used to generate the Java source code. The detailed discussion of the engineering viewpoints and of the technology viewpoints are outside the scope of this paper. Related information can be found in [7].

In RM-ODP Part 3, it is stated that each viewpoint is composed of model elements that are defined in RM-ODP Part 2.

An information viewpoint (of a computational object) is defined as made of information objects. In SEAM we consider that these information objects are modified by "information actions" that represents the computational object's responsibilities. In SEAM, we call these actions localized actions – a term borrowed from Catalysis [8]. For instance, in the example of a supplier value network in Figure 1, the supplier value network information viewpoint represents the products and the customer information as information objects and the exchanges with the customer as a localized action.

A computational viewpoint (of a computational object) is defined as made of computational objects that participate to "computational actions". In SEAM, we call these actions, joint actions; a term also borrowed from Catalysis. For example, in the supplier value network, the computational viewpoint represents the companies (i.e. computational objects described by their information viewpoints) participating to joint actions.

Information viewpoints can be described at different levels of details. This is useful to represent the goal of the systems (typically represented as localized actions or joint actions seen as a whole) or the way the goal is achieved (typically represented as localized actions or joint actions seen as a composite). For instance, the supplier value network has a "Sell" localized action in Figure 2 that represents, as a whole, its capability to sell books. This action can be broken down into "Get Order", "Deliver Book" and "Get Payment" as shown in Figure 3. This composite localized action represents a means to achieve the goal. In SEAM, we call this capability to describe information viewpoints at different levels of details the functional levels.

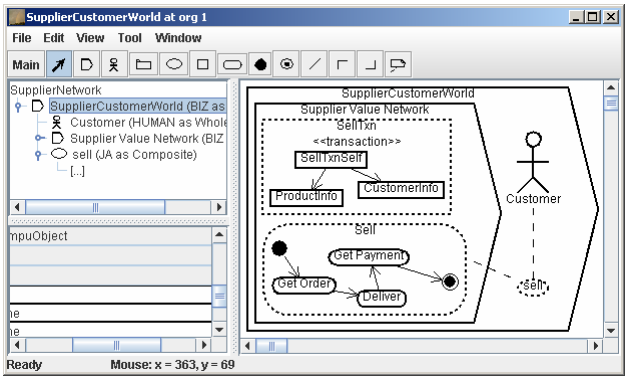

Figure 3. Information viewpoint of computational object "supplier value network" seen at a lower functional level than that in Figure 2

In summary, to define an EA model, it is necessary to have organizational levels that describe the necessary levels (e.g. market, company, departments, IT system, Java programs…). In each organizational level, we have computational objects that represent systems. These computational objects can be described with their information viewpoints (defined as localized actions and information viewpoints) or computational viewpoints (defined as computational objects and joint actions). The information viewpoints can be defined at different functional levels. The problem now consists in understanding how these terms can be defined from RM-ODP Part 2.

2.2. Definition of Model Elements

According to RM-ODP Part 2, an entity is any concrete or abstract thing of interest in the universe of discourse. An entity is represented in the model as a model element. Each model element can be characterized by a basic modeling concept (BMC) and by a specification concept (SC) [9]. Examples of basic modeling elements are: object, action, state. Examples of specification concepts are: composite, types, instances.

The main concept is the computational object. Let's say that a computational object is a kind of object as defined in RM-ODP Part 2. As stated in Part 2, an object has states and actions.

- The state of the computational object is described by information objects that have states. The behavior is described by actions that modify information objects. We call this kind of action localized actions. The localized actions are defined in terms of pre-conditions and post-conditions that change the state of the information objects (the dynamic schema defined in part 3). Additional specification can be captured such as static schemas and invariant schemas that represent the states of the information objects at a given time or the relations between the states of the invariant objects. This description corresponds to the information viewpoint of a computational object.
- The computational objects interact with other computational objects. So, we have another kind of actions that represent the collaboration of a computational object with the other computational objects in its environment. We call this action a joint action. The joint actions change the state of the information objects of the computational objects participating in the collaboration.

In summary, to build an EA model, we have the following basic modeling concepts: computational objects, information objects, localized actions, joint actions and states. It is interesting to highlight that RM-ODP defines the concepts of computational objects, information objects and state. It also defines the concept of actions which groups internal actions and interactions. Our localized action is similar to the RM-ODP concept of action. As RM-ODP considers objects interacting with their environment, they do not need have a concept similar to the joint action. The only concept that might be related is the engineering object that can be used to represent something happening between computational objects. But an engineering object is not an action! We have defined the concept of joint action as it is useful to specify collaboration between multiple objects, seen from a super-observer standpoint. This is an extension we have made to RM-ODP.

As we should be able to describe different functional levels, all information objects, joint actions and localized actions can be represented as whole or as composite. Of course, to be able to represent organizational levels, the computational objects can be represented as whole or as composite as well. This means that we need to have computational objects as whole or as composite, information objects as wholes or as composite, joint actions as wholes or as composite, localized actions as whole or as composite.

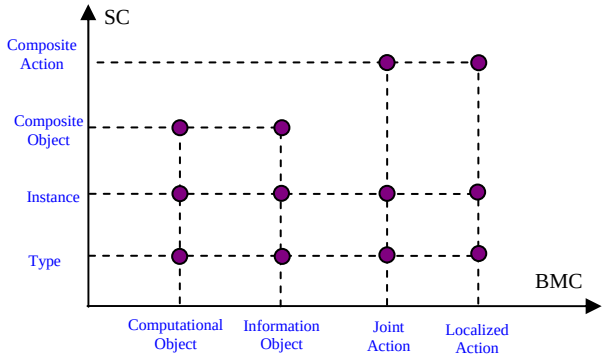

Figure 4. The 16 modeling elements can be defined by combining necessary BMCs and SCs

The model elements in RM-ODP are characterized by a basic modeling concept and by a specification concept such as type or instance. So, to build EA models, we need to combine the necessary BMCs and SCs in order to form needed model elements as discussed above. As illustrated in Figure 4, every BMC can be combined with “Instance” and “Type” to have needed model elements seen as whole. The BMC can also be combined with “Composite Object” or “Composite Action” and then with “Instance” / “Type” to have needed model elements but seen as composite. In summary, the following model elements have been defined in our ontology

- <x> type as a whole
- <x> type as composite
- <x> instance as a whole
- <x> instance as composite

Where <x> is computational object, information object, localized action or joint action.

3. Formalization

Although we consider <x> seen as whole and <x> seen as composite are two separate model elements, we notice that these two model elements always go together and in fact originate from an unique entity in reality. In the EA model, depending on the context, <x> seen as whole or <x> seen as composite is used. To simplify the formalization and thus the design of the CAD tool, we only declare <x>. The design of the CAD tool follows Model-View-Controller approach. Each modeling concept declared in Alloy corresponds to a design class in the part “Model” and is rendered by, nevertheless, two classes in the part “View”: one class for <x> as whole and the other for <x> as composite.

The formalization is done in Alloy 2.0 [3]. Alloy is a lightweight modeling language based on relations and takes the syntax of the first-order logic. Since version 2.0, the language also has object-oriented constructs. The

language is accompanied by a tool called Alloy Constraint Analyzer (ACA) that can be used for simulating and checking models written in Alloy.

The entire Alloy code is given in the appendix at the end of this paper. Note that the Alloy code formalizes the types (<x> type), so all terms in the Alloy could be prefixed with "Type". For clarity reasons, we decided not to do so.

Table 1 shows signature `CompuObject` (`sig` is a keyword in Alloy) that formalizes the computational object. Each computational object optionally mediates a joint action (field `ja`), refers to an information object (field `info`) and a localized action (field `la`), has a number of child computational objects (field `compu_children`) and optionally has a parent computational object (field `compu_parent`). We can interpret that the field `ja` and the field `compu_children` correspond to a computational object seen as composite, whereas the field `compu_parent`, the field `info` and the field `la` stands for that computational object but seen as whole. In other words, the computational object as whole and as composite are formalized in one Alloy signature: `CompuObject`.

Table 1. Alloy code for the computational object

```
sig CompuObject // definition of the computational object
{  // declaration of all fields of the computational object
   ja : option JointAction,
   info : option InfoObject,
   la : option LocalizedAction,
   compu_children : set CompuObject,
   compu_parent : option CompuObject
} { // invariants concerning the computational object
   #compu_parent > 0 => this in compu_parent::compu_children
   all c : compu_children | c::compu_parent = this
   #ja > 0 => ja::compu_med = this
   all j : JointAction - ja | j::compu_med = this =>
          j in ja.^joint_children
   #info > 0 => info::compu_host = this
   all io : InfoObject - info | io::compu_host = this =>
          io in info.^info_children
   #la > 0 => la::compu_owner = this
   all l : LocalizedAction - la | l::compu_owner = this =>
          l in la.^localized_children
}
```

The second block between curly brackets, right after the declaration of all fields of signature `CompuObject`, lists all invariants that must be hold for any computational objects. The first line of this block assures that any computational object must be a child computational object of its parent. The second line implies that all child computational objects have the same parent. The rest of this block concerns the way each computational object relates to information objects, joint actions and localized actions. In short, these invariants mandate the way model elements are related to one another in our ontology.

In EA models, having individual model elements is not enough. Indeed, we need to put them in relation. One way to formalize these relations is to declare additional fields in the signatures that formalize model elements. For example, in signature `CompuObject` we can put a field such as `collaborations : set JointAction` that results in all joint actions in which the declared computational object takes part. Another approach is to declare a particular signature dedicated for each kind of relation. This signature must have at least two fields referencing the source and the destination model element of the declared relation. In this approach, it is easy to put additional attributes such as role name, cardinality… to declared relations.

The entire formalization code is simulated using ACA at the size of 4 (i.e. at most 4 instances shall be created for each signature). Figure 5 shows one possible solution. The signature instances are represented as eclipses. For each instance, the fields are drawn as out-going arrows. The eclipse that an arrow points to represents the value of the field represented by that arrow.

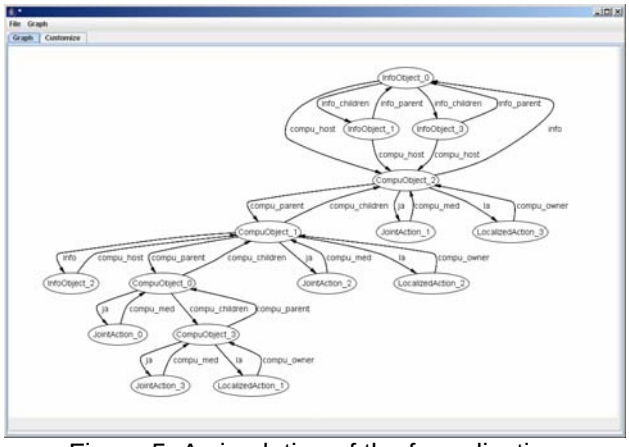

Figure 5. A simulation of the formalization written in Alloy

Table 2 list some correspondences between models of the supplier value network shown in Figure 1, 2, 3 and the Alloy solution shown in Figure 5.

Table 2. Comparison between a model done in our CAD tool and the solution found by ACA

Model Element	Model in CAD tool	Solution by ACA
Computational Object	SupplierCustomerWorld	CompuObject_0
Computational Object	Supplier Value Network	CompuObject_1
Computational Object	Customer	CompuObject_3
Computational Object	Sale Company	CompuObject_2
Joint Action	sell	JointAction_0
Information Object	SellTxn	InfoObject_2
Localized Action	Sell	LocalizedAction_2
Joint Action	sell_collaboration	JointAction_2

4. Example of UML Representation

The information viewpoint of the Supplier Value Network can be filtered to show only the class diagram (Figure 6 a) or the activity diagram (Figure 6 b). The modeler can easily do so while selecting the pictogram of the Supplier Value Network in the diagram illustrated in Figure 3.

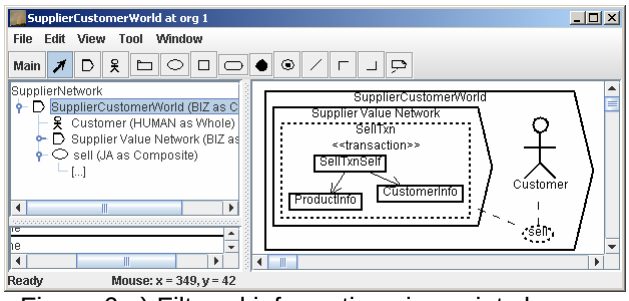

Figure 6 a) Filtered information viewpoint shows a class diagram of “supplier value network”. This class diagram is shown in its context.

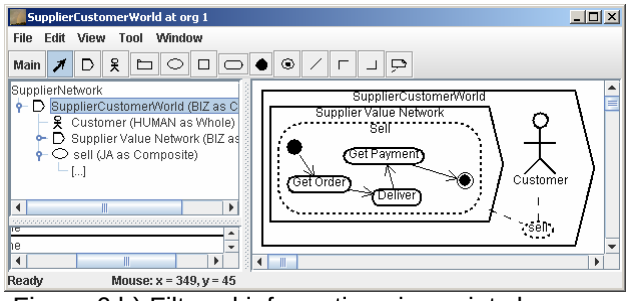

Figure 6 b) Filtered information viewpoint shows an activity diagram of “supplier value network”. This activity diagram is shown in its context.

The value of this representation lies in the fact that the modeler can ultimately reason about each system by using individual UML diagrams. Future versions of our tool will allow her to edit these diagrams in separate windows, making it more UML-compatible.

5. Related Work

RM-ODP based modeling has increasingly attracted researchers in the field of EA and formal methods. [10] proposes a tool environment for viewpoint-oriented EA. This framework allows the modeler to define various viewpoints (that are not necessarily limited to RM-ODP viewpoints) of an EA project. [11] aims at relating RM-ODP enterprise viewpoint and computational viewpoint to make consistency checking feasible. [12] explores the possibility of formally writing computational viewpoint specifications in Maude, an object-oriented executable rewriting logic language. [13] proposes a declarative language (with specialized constructs for security) to specify enterprise viewpoints. [14] describes a formal interpretation to viewpoint consistency and proposes some strategies for checking viewpoint consistency in general. [15] focuses on the consistency between the computational viewpoint and the engineering viewpoint. Our work addresses hierarchical systems in EA and tries to integrate the information viewpoint and the computational viewpoint of RM-ODP to make a computer-interpretable tool-supported ontology with quite precise semantics of model elements. We use Alloy, a lightweight modeling language based on set theory with object-oriented constructs, to formalize our ontology.

One of the big concerns about RM-ODP viewpoints is the transition between them. [16] defines information actions and proposes the articulation between them and messages in computational view. [17] establishes one-to-one mapping from an information action to a stateless object. In our ontology, information objects representing the transactions are similar to stateless objects [17]. We believe that, for a certain computational object, its composite view is quite dependent from its whole view. The decomposition made on a composite computational object is just a choice of the modeler, although it can be inspired by the whole view. Our tool manages the connection between the two views. However, it leaves the decomposition to the modeler as her own decision.

Conclusions

We seek for an ontology that is suited to modeling hierarchical systems in EA. We base our work on RM-ODP. To define the necessary model elements and their relations in the ontology, we integrate the computational viewpoint and the information viewpoint of RM-ODP. The resulting ontology is computer-interpretable and is implemented in a CAD tool. It is also formalized in Alloy and simulated using Alloy Constraint Analyzer. The CAD tool allows the modeler to build models for hierarchical systems where every model element can be seen either as whole or as composite. This tool can also render UML diagrams according to some filtering option to reflect particular aspects of the system of interest.

Our future work investigates the possibility to translate the formalization written in Alloy to partial Java code (mainly in the “Model” part of the entire Model-View-Controller design of the tool) that implements our CAD tool so that we can achieve some automation between the ontology specification and the ontology implementation. We also consider developing advanced features that allows the modeler to separately edit UML diagrams of the system of interest.

References

[1] A. Wegmann, "On the Systemic Enterprise Architecture Methodology (SEAM)", in the proceedings of ICEIS 2003, Angers, France, 2003.

[2] OMG, "ISO/IEC 10746-1, 2, 3, 4 | ITU-T Recommendation, X.901, X.902, X.903, X.904, Reference Model of Open Distributed Processing," 1995-1996.

[3] MIT, The Alloy Constraint Analyzer, http://alloy.mit.edu/

[4] A. Naumenko, Wegmann, A., Genilloud, G., Frank, W.F., "Proposal for a formal foundation of RM-ODP concepts", in the proceedings of ICEIS/WOODPECKER 2001, Setúbal, Portugal, 2001.

[5] A. Naumenko, Wegmann, A., "A Metamodel for the Unified Modeling Language", in the proceedings of <<UML>> 2002, Dresden, Germany, 2002.

[6] L. S. Lê, Wegmann, A., "SeamCAD 1.x: User's Guide," School of Computer and Communication Sciences, EPFL, Lausanne IC/2004/98, November 2004 2004.

[7] A. Wegmann, Preiss, O., "MDA in Enterprise Architecture? The Living SystemTherory to the Rescue…", in the proceedings of 7th IEEE EDOC, Brisbane, Australia, 2003.

[8] D. Francis D'souza, Cameron Wills, A., *Object, Components and Frameworks with UML, The Catalysis Approach*: Addison-Wesley, 1999.

[9] A. Wegmann, Naumenko, A., "Conceptual Modeling of Complex Systems using an RM-ODP based Ontology", in the proceedings of 5th IEEE EDOC, Seattle, USA, 2001.

[10] M. W. A. Steen, Akehurst, D.H., ter Doest, H.W.L., Lankhorst, M.M., "Supporting Viewpoint-Oriented Enterprise Architecture", in the proceedings of 8th IEEE EDOC, California, USA, 2004.

[11] R. Dijkman, Quartel, D., Pires, L., Sinderen, M., "A Rigorous Approach to Relate Enterprise and Computational Viewpoints", in the proceedings of 8th IEEE EDOC, California, USA, 2004.

[12] R. Romero, Vallecillo, A., "Formalizing ODP Computational Viewpoint Specifications in Maude", in the proceedings of 8th IEEE EDOC, California, USA, 2004.

[13] E. Lupu, Sloman, M., Dulay, N., Damianou, N., " Ponder: realising enterprise viewpoint concepts", in the proceedings of 4th IEEE EDOC, Makuhari, Japan, 2000.

[14] H. Bowman, Steen, M., Boiten, E., Derrick, J., "A Formal Framework for Viewpoint Consistency", *Formal Methods in System Design*, 2002.

[15] E. Boiten, Bowman, H., Derrick, J., Linington, P., Steen, M., "Viewpoint consistency in ODP", *ELSEVIER*, 2000.

[16] J. Dustzadeh, Najm, J., "Consistent Semantics for ODP Information and Computational Models", in the proceedings of FORTE/PSTV'97, Osaka, Japan, 1997.

[17] C. Bernardeschi, Dustzadeh, J., Fantechi, A., Najm, J., Nimour, A., Olsen, F., "Transformation and Consistent Semantics for ODP Viewpoints", in the proceedings of FMOODS'97, Canterbery, UK, 1997.

Appendix: Ontology Formalization in Alloy

```
module seamcad

sig CompuObject
{
   ja : option JointAction,
   info : option InfoObject,
   la : option LocalizedAction,
   compu_children : set CompuObject,
   compu_parent : option CompuObject
} {
   #compu_parent > 0 => this in compu_parent::compu_children
   all c : compu_children | c::compu_parent = this
   #ja > 0 => ja::compu_med = this
   all j : JointAction - ja | j::compu_med = this => j in ja.^joint_children
   #info > 0 => info::compu_host = this
   all io : InfoObject - info | io::compu_host = this => io in info.^info_children
   #la > 0 => la::compu_owner = this
   all l : LocalizedAction - la | l::compu_owner = this => l in la.^localized_children
}

fact compu_acyclic {
   all c : CompuObject | (c !in c.^compu_parent && c !in c.^compu_children)
}

sig InfoObject
{
   compu_host : CompuObject,
   info_children : set InfoObject,
   info_parent : option InfoObject
} {
   all c : info_children | c::info_parent = this && c::compu_host = this::compu_host
   #info_parent > 0 => this in info_parent::info_children
}

fact info_acyclic {
   all c : InfoObject | (c !in c.^info_parent && c !in c.^info_children)
}

sig JointAction {
   compu_med : CompuObject,
   joint_children : set JointAction,
   joint_parent : option JointAction
} {
```

```
   all c : joint_children | c::joint_parent = this && c::compu_med = this::compu_med
   #joint_parent > 0 => this in joint_parent::joint_children
}

fact joint_acyclic {
   all c : JointAction | (c !in c.joint_parent && c !in c.^joint_children)
}

sig LocalizedAction {
   compu_owner : CompuObject,
   localized_children : set LocalizedAction,
   localized_parent : option LocalizedAction
} {
   all c : localized_children | c::localized_parent = this && c::compu_owner = this::compu_owner
   #localized_parent > 0 => this in localized_parent::localized_children
}

fact localized_acyclic {
   all c : LocalizedAction | (c !in c.^localized_parent && c !in c.^localized_children)
}

sig IO2IO { // association
   source : InfoObject,
   destination : InfoObject
} {
   source::compu_host = destination::compu_host
}

sig LA2LA { // action transition
   source : LocalizedAction,
   destination : LocalizedAction
} {
   source::localized_parent = destination::localized_parent
   source::compu_owner = destination::compu_owner
}

sig JA2JA { // action trnasition
   source : JointAction,
   destination : JointAction
} {
   source::joint_parent = destination::joint_parent
   source::compu_med = destination::compu_med
}

sig CO2JA { // collaboration link
   source : CompuObject,
   destination : JointAction
} {
   source::compu_parent = destination::compu_med
}

sig JA2CO { // collaboration link
   source : JointAction,
   destination : CompuObject
} {
   source::compu_med = destination::compu_parent
}

sig IO2CO { // trace dependency
   source : InfoObject,
   destination : CompuObject
}

fact oneRoot {
   sole co : CompuObject | #co.compu_parent = 0
}

fun hasLifecycle() {
   some co : CompuObject | one co.ja && no co.la && no co.info && #co.compu_children = 2
}

run hasLifecycle for 4
```